

CROSS-BORDER REMITTANCE TRACK

RemitSafe

Multi-Asset Payment Pledge System

A blockchain-backed OFW payment pledge platform using Native ETH and ERC-20 stablecoins (USDC/USDT) on Morph L2.



ETH

Native escrow



USDC

ERC-20 stablecoin



USDT

ERC-20 stablecoin

CORE IDEA

Turn remittance promises into verifiable multi-asset commitments

BLOCKCHAIN

Morph L2 - EVM-compatible, Holesky Testnet

BUILD PERIOD

May 18 - May 29, 2026

HACKATHON

Build In! Payments - Morph x Blockchain4Youth x DVCode



Table of Contents

1. Project Overview
 2. Problem Statement
 3. Solution: Multi-Asset RemitSafe
 4. End-to-End User Scenario
 5. Web and Mobile App Scope
 6. System Architecture
 7. Tech Stack and Languages
 8. Multi-Asset Smart Contract Design
 9. KYC, Facial Recognition, and Login Recovery
 10. On-Chain vs Off-Chain Data
 11. Reputation Score System
 12. Testing Strategy
 13. Phase Development Plan
 14. Submission Checklist
 15. 200-Word Write-Up
 16. Security and Logic Issues
 17. Future Roadmap
-

1. Project Overview

RemitSafe is a blockchain-backed OFW payment pledge system built on Morph L2. The updated version supports three payment assets: Native ETH, USDC, and USDT. This makes the prototype flexible for hackathon testing while still showing a realistic remittance path using stablecoins.

Instead of only sending money immediately, a sender can lock partial funds in escrow and commit to a remaining payment date. The merchant can view the pledge on-chain and decide whether to accept the sender based on collateral, deadline, and wallet reputation.

Native ETH Support

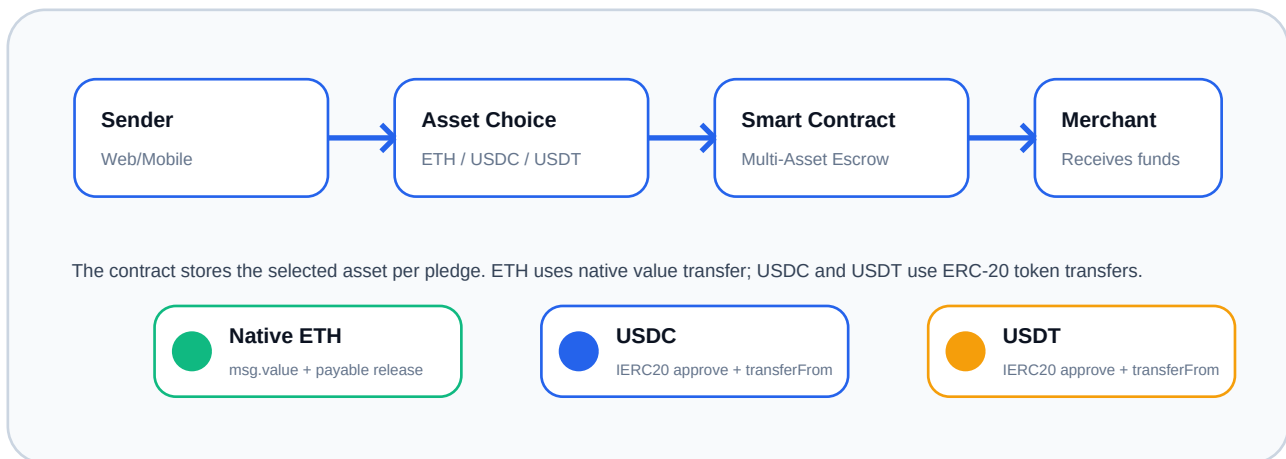
Fastest for demo. Sender deposits ETH directly using payable functions and Morph Holesky test ETH.

USDC Support

Realistic digital dollar option. Requires ERC-20 approve and transferFrom before escrow.

USDT Support

Another stablecoin option for remittance. Also handled as ERC-20 through an allowlisted token address.



Recommended hackathon positioning: RemitSafe is not just a remittance transfer app. It is a trust layer for future payments, powered by multi-asset smart contract escrow, identity verification, and on-chain reputation.

2. Problem Statement

Problem	Why It Matters
Timing Gap	OFW salary dates often do not match Philippine due dates for tuition, rent, medical bills, or family obligations.
No Commitment Layer	Traditional remittance apps can transfer money, but they cannot create a verifiable promise to pay later.
Merchant Trust Problem	Schools, landlords, and clinics have no reliable proof that the sender will complete a future payment.
Currency Flexibility	Some users prefer ETH for testnet or crypto-native flows, while others prefer USDC/USDT stablecoins for predictable value.
Limited Financial History	OFWs may have income abroad but little local credit history in the Philippines. RemitSafe can build behavior-based reputation.

3. Solution: Multi-Asset RemitSafe

RemitSafe introduces a Payment Pledge: a smart contract-enforced commitment where the sender locks a partial amount today and pledges to complete the balance on a future date. The new design supports both native and tokenized assets.

Asset	Technical Handling	Best Use
ETH	Native coin. createPledgeETH() and depositRemainingETH() use msg.value.	Fast testnet demo and simple wallet flow.
USDC	ERC-20 token. User approves the contract, then the contract pulls funds via transferFrom.	Stable-value remittance demo.
USDT	ERC-20 token. Same token workflow as USDC with a separate allowlisted token address.	Alternative stablecoin support.



4. End-to-End User Scenario

Step	Scenario
1. Tuition Due	Juan needs to pay school tuition, but payday is next week. He opens the RemitSafe mobile app.
2. Asset Selection	Juan chooses ETH, USDC, or USDT. For stable value, he may choose USDC/USDT; for demo speed, he may choose ETH.
3. Pledge Creation	Juan locks a 20% upfront deposit in smart contract escrow and sets the school wallet as merchant recipient.
4. Merchant Verification	The school dashboard shows locked collateral, selected asset, remaining amount, due date, KYC status, and reputation score.
5. Remaining Deposit	On payday, Juan deposits the remaining balance. The contract verifies full funding and automatically releases the total payment.
6. Default Handling	If Juan misses the deadline and grace period, the merchant can claim the locked partial amount and Juan reputation is reduced.

5. Web and Mobile App Scope

The project should be presented as a web application plus a mobile application. The web app is ideal for merchants and admin dashboards. The mobile app is ideal for OFW senders who need quick onboarding, biometric login, and pledge management.

Platform	Main Users	Core Screens
Web App	Merchants, admin, demo judges	Merchant dashboard, pledge verification, claim partial, KYC review, analytics.
Mobile App	OFW senders and family users	Registration, government ID upload, face scan, create pledge, deposit balance, reminders, recovery.
Shared Backend	All users	KYC status, user profiles, notifications, off-chain metadata, event indexing.



6. System Architecture

RemitSafe uses a hybrid architecture: on-chain escrow and reputation for trust, off-chain profile and identity data for privacy, and app-layer dashboards for usability.

Layer	Technology	Responsibility
Web Frontend	Next.js 14, TypeScript, Tailwind CSS, ethers.js	Merchant dashboard, admin dashboard, wallet connection, pledge visibility.
Mobile App	React Native or Expo	Sender onboarding, face scan flow, pledge creation, reminders, login recovery.
Backend/API	Next.js API Routes or Node.js API	User profiles, notification jobs, event indexing, KYC status, app metadata.
Database	PostgreSQL / Supabase	Off-chain user data, KYC status, document references, audit logs.
Storage	Supabase Storage / S3	Encrypted government ID images and selfie/liveness artifacts.
Blockchain	Morph L2 + Solidity	ETH escrow, ERC-20 escrow, release logic, default logic, reputation score.
AI / Identity	Face matching + liveness detection service	Compare selfie to ID photo, prevent duplicate/fake accounts, improve login security.

Design note: government ID and face images must not be stored on-chain. Only a verification status, timestamp, and optional hash reference should be stored off-chain.



7. Tech Stack and Languages

Category	Tools / Languages	Purpose
Smart Contract	Solidity 0.8.x, OpenZeppelin, ReentrancyGuard, Ownable/Pausable	Multi-asset pledge contract and secure escrow logic.
Token Support	Native ETH + IERC20 for USDC/USDT	ETH uses payable. USDC/USDT use ERC-20 approve and transferFrom.
Blockchain Tooling	Hardhat, ethers.js v6, Morph Holesky	Compile, test, deploy, and interact with smart contracts.
Web App	Next.js 14, TypeScript, Tailwind CSS	Dashboard, API routes, responsive UI.
Mobile App	React Native / Expo	Mobile-first account creation, face recognition flow, push notifications.
Database	PostgreSQL / Supabase	KYC status, user profiles, pledge metadata, notification logs.
Identity Layer	Face recognition, liveness detection, government ID capture	Account creation, login verification, GCash-like recovery.
Hosting	Vercel, Supabase, optional Railway/Render	Deploy web, API, database, and service workers.

Important clarification: USDC and USDT are ERC-20 tokens. ETH is not ERC-20 when used as native ETH. If the team later uses WETH, that version would be ERC-20.

8. Multi-Asset Smart Contract Design

Function	Asset Type	Description
createPledgeETH()	Native ETH	Sender creates pledge by sending msg.value as upfront collateral.
createPledgeToken()	USDC / USDT ERC-20	Sender approves token first, then contract transfers upfront amount into escrow.
depositRemainingETH()	Native ETH	Sender deposits remaining ETH before deadline or within grace period.
depositRemainingToken()	USDC / USDT ERC-20	Sender deposits remaining stablecoin amount using ERC-20 transferFrom.
releaseFunds()	All assets	Internal release after full funding. ETH uses call; tokens use SafeERC20.safeTransfer.
claimPartial()	All assets	Merchant claims upfront collateral after deadline plus grace period.
refundSender()	All assets	Sender refunds collateral if merchant misses claim window.
getPledge()	All assets	Returns asset type, token address, amounts, deadlines, status, sender, and merchant.

Suggested enum model:

```
enum AssetType { NATIVE_ETH, ERC20_TOKEN }
```

For ERC-20 pledges, store tokenAddress and allow only approved stablecoin addresses. For native ETH pledges, tokenAddress should be address(0).



9. KYC, Facial Recognition, and Login Recovery

RemitSafe should include identity verification because payment pledges rely on trust. The sender account creation flow can require a valid government ID, selfie capture, face matching, and liveness check before high-value pledges are allowed.

Feature	Purpose	Implementation Notes
Government ID Upload	Verify the user real identity before pledge creation.	Store encrypted image off-chain; never store ID images on-chain.
Face Recognition	Match selfie against ID photo during registration and risky login.	Use a face embedding comparison service or API. Store only verification result and reference hash.
Liveness Detection	Prevent printed-photo or screen replay attacks.	Blink, head-turn, or random challenge during selfie capture.
GCash-like Recovery	Allow secure account recovery for lost phone/password.	Use OTP, face verification, ID re-check, and cooldown period.
Risk-Based Login	Require face scan only for sensitive actions.	Example: new device, large pledge, wallet change, or recovery request.

Privacy rule: facial data and government ID data are sensitive. They must remain off-chain, encrypted, access-controlled, and removable under the project privacy policy.



10. On-Chain vs Off-Chain Data

Data	Location	Reason
ETH escrow amount	On-chain	Core financial commitment; must be trustless and auditable.
USDC/USDT escrow amount	On-chain	Token balances and releases must be contract-enforced.
Asset type and token address	On-chain	Merchant must verify what asset is pledged.
Pledge deadline and status	On-chain	Core commitment and default logic.
Reputation score	On-chain	Public trust signal for merchants.
Name, phone, email	PostgreSQL	Private user data; not appropriate for public chain.
Government ID image	Encrypted storage	Sensitive document; store off-chain only.
Face verification result	Database	Store pass/fail and timestamp, not raw biometric data on-chain.
Notification logs	Database	Operational data for reminders and audit trail.

11. Reputation Score System

The reputation score helps merchants evaluate whether a sender is reliable. It should be attached to the sender wallet and updated by the contract after completed, late, or defaulted pledges.

Suggested formula: $(\text{On-time pledges} \times 10) - (\text{Defaults} \times 25) - (\text{Late payments} \times 5) + \text{Volume bonus}$. Use int256 or bounded subtraction to avoid underflow.



12. Testing Strategy

Test Area	Expected Result
ETH pledge creation	Contract receives msg.value and records assetType as NATIVE_ETH.
USDC pledge creation	User approval succeeds; contract pulls USDC and records token address.
USDT pledge creation	User approval succeeds; contract pulls USDT and records token address.
Unsupported token	Contract rejects token not in allowlist.
Deposit remaining	Full pledge amount triggers automatic release to merchant.
Grace period	Sender can still complete pledge before grace period ends.
Default claim	Merchant can claim partial amount only after deadline plus grace period.
Refund	Sender can recover collateral if merchant misses claim window.
KYC onboarding	Registration requires ID upload, face match, and liveness result.
Recovery flow	Account recovery requires OTP plus face re-verification before account access.



13. Phase Development Plan

Phase	Days	Deliverables
1. Setup and Design	May 18-19	Initialize repo, Next.js, Hardhat, database schema, mobile app skeleton, UI design.
2. Multi-Asset Contract	May 20-21	Implement ETH pledge flow, ERC-20 pledge flow, allowlist, default logic, unit tests.
3. Backend and KYC	May 22-23	API routes, user profiles, KYC status, encrypted storage references, notification jobs.
4. Web and Mobile UI	May 24-25	Sender mobile flow, merchant web dashboard, wallet connection, asset selection.
5. Integration Testing	May 26	End-to-end pledge, deposit, release, default, KYC onboarding, and recovery test.
6. Polish and Diagrams	May 27	Architecture diagram, smart contract flow, demo script, UI cleanup.
7. Submission	May 28-29	Demo video, public URL, GitHub repository, 200-word write-up, build diary posts.

14. Submission Checklist

Required Item	Status Target
Public GitHub repository	Includes web, mobile, contracts, README, and deployment guide.
Working demo URL	Vercel deployment for web dashboard and API routes.
Morph Holesky deployment	Contract address visible on Morph Explorer.
3-minute demo video	Problem, live pledge demo, KYC/mobile flow, future plan.
200-word write-up	Clear explanation of multi-asset RemitSafe.
Build diary posts	Use official hackathon hashtags and show progress.

15. 200-Word Write-Up

Millions of OFWs face a painful timing gap: bills in the Philippines are due before payday abroad. Traditional remittance apps only move money immediately; they do not provide a trusted promise to pay later. RemitSafe solves this through blockchain-backed payment pledges on Morph L2. A sender can choose ETH, USDC, or USDT, lock a partial amount into smart contract escrow, and commit to a future deposit date for the remaining balance. Merchants such as schools, landlords, and clinics can verify the pledged asset, locked collateral, deadline, and sender reputation directly from the blockchain. When the sender completes the remaining payment, the contract automatically releases the full amount to the merchant. If the sender defaults after the grace period, the merchant can claim the locked partial amount and the sender reputation score is updated. RemitSafe also adds identity trust through web and mobile onboarding, government ID verification, facial recognition, liveness detection, and GCash-like account recovery. Sensitive identity data stays off-chain in encrypted storage, while financial commitments remain transparent and auditable on-chain. The result is a flexible multi-asset remittance pledge system that combines blockchain escrow, stablecoin support, identity verification, and behavior-based financial reputation for OFWs.

16. Security and Logic Issues

Issue	Risk	Fix
ETH transfer reentrancy	A malicious recipient contract could re-enter during release.	Use ReentrancyGuard and update state before external transfer.
ERC-20 transfer quirks	Some tokens do not return standard boolean values.	Use OpenZeppelin SafeERC20.
Unsupported tokens	Users could pledge fake tokens with no real value.	Use token allowlist for USDC and USDT addresses.
Wrong asset release	Contract could mix ETH and token logic.	Store assetType and tokenAddress per pledge; route release by asset type.
Zero or weak collateral	Sender could create meaningless pledge.	Require minimum upfront deposit, e.g., 20%.
Long deadlines	Sender could set unrealistic payment date.	Limit commitment date, e.g., max 90 days.
Identity privacy leak	Government ID or face data could be exposed.	Keep PII off-chain, encrypted, and access-controlled.
Reputation underflow	Unsigned score could wrap if default penalty is larger than score.	Use int256 or bounded subtraction.

Recommended contract libraries: OpenZeppelin ReentrancyGuard, SafeERC20, Ownable, and Pausable for production readiness.

17. Future Roadmap

Stage	Enhancements
After Submission	Improve UI polish, add stronger mobile screens, complete README, publish architecture and flow diagrams.
Code Review Period	Verify Morph deployment, retest ETH/USDC/USDT flows, prepare answers for judges.
Post-Hackathon	Deploy to Morph mainnet, integrate real stablecoin contracts, improve KYC provider integration.
Philippines Expansion	Explore GCash/Maya merchant settlement, remittance center partnerships, and bank/cooperative credit scoring API.
Advanced Identity	Add duplicate-account detection, risk scoring, and privacy-preserving verification proofs.

Final Positioning

RemitSafe is a web and mobile multi-asset pledge system for OFW remittance commitments. ETH makes the prototype easy to demo. USDC and USDT make the concept realistic for stable-value payments. Facial recognition and government ID verification add identity trust, while smart contracts add financial trust.
